

UCS645

PARALLEL & DISTRIBUTED COMPUTING



ASSIGNMENT 6

Submitted by: Ritwik Gupta (102303096)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Task A: CUDA Device Query

1. GPU Architecture and Compute Capability

The NVIDIA GeForce MX130 utilizes the **Maxwell** architecture (inferred from the version number). Its compute capability is **5.0**.

2. Maximum Block Dimensions

For the MX130, the maximum dimensions of a block are **(1024, 1024, 64)** for the x, y, and z axes respectively. However, the total number of threads combined within a single block cannot exceed **1024**.

3. Maximum Threads Calculation

The question poses a specific hypothetical scenario: "If the hardware's maximum grid dimension is 65535 and the maximum block dimension is 512". Based on those specific constraints: $\text{Maximum Threads} = 65535 \times 512 = 33553920$ threads.

4. Reasons to Not Launch Maximum Threads

A programmer might intentionally avoid launching the maximum number of threads due to:

- **Resource Exhaustion:** Each thread consumes registers and shared memory. If a block requests more resources than the Streaming Multiprocessor (SM) can provide, the kernel launch will fail or performance will severely degrade.
- **Data Size:** If the dataset is smaller than the maximum thread count, spawning excess threads wastes scheduling overhead.
- **Memory Bandwidth Bottlenecks:** Launching too many threads might saturate the memory bus, leading to cache thrashing rather than useful computation.

5. Limitations on Launching Maximum Threads

The hardware imposes strict limits per Streaming Multiprocessor (SM). A program is limited by:

- The maximum number of registers available per block/SM.
- The amount of shared memory dynamically or statically allocated per block.
- The hard limit of 1024 threads per block.

6. Shared Memory

Shared memory is exceptionally fast, user-managed on-chip memory that is shared among all threads within a single block. It is used to cache data locally and avoid expensive global memory accesses.

- **MX130 Capacity:** Your GPU has **48 KB** of shared memory per block.

7. Global Memory

Global memory is the main Video RAM (VRAM) of the GPU. It is accessible by all threads across all blocks and the host CPU, but it has the highest latency and lowest bandwidth of all GPU memory types.

- **MX130 Capacity:** Your GPU has **1.95 GB** of total global memory.

8. Constant Memory

Constant memory is a specialized, read-only memory space that is aggressively cached. It is highly efficient when all threads in a warp read the exact same memory address simultaneously.

- **MX130 Capacity:** Your GPU has **64 KB** of constant memory.

9. Warp Size Significance

A warp is the fundamental unit of execution in NVIDIA GPUs. The hardware schedules and executes instructions at the warp level, meaning all threads in a warp execute the same instruction in lockstep (SIMT execution).

- **MX130 Warp Size:** Your GPU's warp size is **32 threads**.

10. Double Precision Support

Yes, double precision (FP64) is supported on the MX130, as compute capability 5.0 natively supports it. However, on consumer Maxwell chips like this one, FP64 performance is heavily throttled compared to single-precision (FP32) computation.

Task B: Array Sum in CUDA

1. Objective

To compute the total sum of a large array containing single-precision floating-point numbers by leveraging GPU parallelization.

2. Problem Description

The task requires implementing a complete CUDA workflow to sum the elements of an array.

The required steps include:

1. Allocating memory on the device (GPU).
2. Copying the generated input array from the host (CPU) to the device.
3. Initializing appropriate thread block and kernel grid dimensions.
4. Invoking the CUDA kernel to compute the sum.
5. Copying the final computed result back to the host.
6. Freeing the allocated device memory.

3. Methodology

- **Memory Management:** Device memory was allocated using `cudaMalloc` for the array and a single variable to store the final result. Data was transferred using `cudaMemcpyHostToDevice`.
- **Execution Configuration:** A block size of 256 threads was chosen, and the grid size was dynamically calculated based on the total array size N using $(N+255)/256$.
- **Kernel Design:** * The kernel utilizes shared memory (`__shared__`) to load elements efficiently.
 - A tree-based parallel reduction is performed within each block, halving the active threads at each step to sum the local values.
 - To prevent race conditions across different blocks writing to the final global sum, Thread 0 of each block uses `atomicAdd` to safely update the global result.
- **Cleanup:** Memory was released back to the system using `cudaFree`.

4. Conclusion

The array sum was successfully computed. Using shared memory for block-level reduction significantly decreases the number of slow global memory accesses, and `atomicAdd` provides a safe, synchronization-free way to aggregate the final block sums.

Task C: Integer Matrix Addition

1. Objective

To perform parallel matrix addition on two large integer matrices using CUDA and analyze the computational and memory access overhead.

2. Problem Description

The kernel computes the element-wise sum of two 2D integer matrices, A and B, storing the result in matrix C, such that $C[i][j] = A[i][j] + B[i][j]$. A 2D grid and 2D block configuration are used to map threads to the matrix indices.

3. Performance Analysis

Based on the kernel execution for matrices of dimensions $N \times M$, the operational metrics are as follows:

- **Floating-Point Operations (FLOPs): 0 operations.** Since the matrices are explicitly defined as *integer* matrices, the addition operation uses the Integer Arithmetic Logic Unit (ALU), not the Floating-Point Unit (FPU). (Note: There are exactly $N \times M$ integer additions performed).
- **Global Memory Reads: $2 \times N \times M$ reads.** Every single thread computing a value must read exactly one integer from matrix A and one integer from matrix B.
- **Global Memory Writes: $N \times M$ writes.** Every single thread writes exactly one integer result to the corresponding index in matrix C.

4. Conclusion

Matrix addition is a classic example of a **memory-bound** algorithm. The ratio of computation (one integer addition) to memory access (two reads, one write) is extremely low. Therefore, the overall performance of this kernel on the GPU is strictly limited by the global memory bandwidth rather than the core compute capability.